Gamma and Beta Random Number Generators

Jayden Gould, Harrison Funk, Elijah DiFuria, Kian Jadbabaei

2026-02-18

Introduction

We selected the article "Acceptance-Rejection Techniques for Sampling from the Gamma and Beta Distributions," authored by U. Dieter and J.H. Ahrens in 1974. The paper builds upon John von Neumann's early concepts of generating random variables by majorizing a target probability density function.

Summary of Article and its Significance to RNG

Generating random variables in computational statistics often relies on the Inverse Cumulative Distribution Function method. However, for continuous and non-uniform distributions like the Gamma function, it is often impractical because of the lack of a closed-form inverse CDF. To solve this, U. Dieter and J. H. Ahrens (1974) applied the Acceptance-Rejection methodology, which bypasses the direct simulation of a target density $f(x)$ by utilizing a simpler proposal envelope $g(x)$ where $g(x) \geq f(x)$.

The primary significance of the article lies in its resolution of the computational bottlenecks. The bottlenecks usually occur when the distribution shape parameters scale. The authors developed algorithms with remarkably flat execution times, regardless of parameter size. The algorithms abandon loose, universal envelopes in favor of highly tailored, optimized majorizing functions (such as the Cauchy distribution and the Normal-Exponential split). Furthermore, they pioneered the use of algebraic "squeeze" bounds, which are simple polynomials that quickly accept or reject samples, which bypass the need for expensive transcendental function evaluations (such as logarithms or exponentials) in over 99% of simulation loops. Today, variations of these exact algorithms remain actively embedded in the source code of foundational statistical software, including R's internal rgamma() function.

Detailed Description of the RNG Methods Covered

For this project, we discovered a mathematical constraint identified by Dieter and Ahrens: no single acceptance-rejection envelope can correctly cover the entire domain of the Gamma shape parameter $a \in (0, \infty)$. To build a universally functional Gamma generator, we implemented a wrapper function of three algorithms from the text:

Algorithm GS ($a \leq 1$): When $a \leq 1$, the Gamma density has an asymptote at zero, making standard envelope construction difficult. Algorithm GS bypasses this by creating a piecewise envelope $g(x)$ proven in Lemma 4 of the text. For the core region ($0 \leq x \leq 1$), it utilizes a power function proposal, generating candidates via the inverse transform $x = p^{1/a}$ where $p = bu$ and $b = (e + a)/e$. For the tail region ($x > 1$), it switches to an exponential proposal generated via $x = -\ln((b - p)/a)$. Each candidate then is subject to a log-scale acceptance test, and the paper proves the expected number of trials per accepted sample stays between 1.0 and 1.4 across all valid a, making GS highly efficient.

Algorithm GC ($1 < a \leq 2.53$): For parameters slightly above 1, we implemented the Cauchy Method, derived from Lemma 1 of the text. Since the Cauchy distribution has polynomial tails that decay at $O(x^{-2})$ rather than exponentially, it stays thick enough in the tails to sit above the Gamma density, making it a valid envelope for this parameter range. The envelope is optimally centered at the Gamma's mode $b = a - 1$ and scaled by $s = \sqrt{2a - 1}$, chosen so that the majorizing function touches the Gamma target exactly at its

mode. Candidates are drawn via the Cauchy inverse CDF:

$$x = b + s \cdot \tan\left(\pi\left(u - \tfrac{1}{2}\right)\right)$$

and acceptance is determined by a single log-scale ratio evaluation. There are no squeeze bounds in this algorithm; the paper reports that the expected number of trials decreases from $\pi \approx 3.14$ at $a = 1$ toward $\sqrt{\pi} \approx 1.77$ as $a \to \infty$, so efficiency improves as the parameter grows. The upper bound of $a \leq 2.53$ is not mandatory, it is the threshold below which Algorithm GO's squeeze bounds are mathematically invalid, as proven through a cubic discriminant analysis in Lemma 3.

Algorithm GO ($a > 2.53$): For larger parameters, the Gamma density is well-approximated by a normal distribution centered at its mode $\mu = a - 1$, making a normal envelope practical. However, it can be shown that no Gaussian curve can fully enclose a Gamma density since the right tail will always exceed any normal envelope. Algorithm GO implements a Normal-Exponential split: a Normal distribution envelope with mean $\mu = a - 1$ and standard deviation

$$\sigma = \sqrt{a + \sqrt{\tfrac{8}{3}} \cdot \sqrt{a}}$$

covers the core region $(0, b)$, while an Exponential envelope covers the extreme right tail $[b, \infty)$. The tail region is sampled from with a fixed probability $\beta = 0.009572\ldots$ rather than computing the exact ratio $H/(G + H)$ at every call. The authors showed that this ratio is bounded above by β across all values of a, so replacing it with a single constant guarantees the same correctness at a far smaller cost. Algorithm GO introduces the "Quick Acceptance'' squeeze bounds from Lemma 3, which are simple polynomial lower bounds on the acceptance ratio $f(x)/g(x)$ that allow the vast majority of proposals to be accepted or rejected without any logarithm evaluations. Rather than computing the expensive log-density ratio at every step, the squeeze bounds work by checking whether u falls below a cheap polynomial approximation of $f(x)/g(x)$ first. For a proposed sample x, let $s = (x - \mu)/\sigma$ be its standardized deviate, $S = s^2/2$ its half squared value, and $W = \sigma^2/\mu$ a shape-dependent variance ratio. For $s \geq 0$, the bound is:

$$L = 1 - S(W - 1)$$

and for $s < 0$, a cubic correction term is added:

$$L = 1 - S(W - 1) + \frac{s^3}{\sqrt{a}} \cdot W$$

The full log-density test is only reached in the rare cases where neither squeeze fires.

The Gamma-Ratio Beta Generator: To further extend the utility of our code, we implemented the theorem detailed on page 3 of the text to generate Beta distributions. Because a true $\beta(a, b)$ distribution can be formed by the ratio

$$\frac{x}{x + y}$$

where x and y are independent variables drawn from $\gamma(a)$ and $\gamma(b)$ respectively, we constructed a Beta generator that relies exclusively on our optimized Gamma function. The paper specifically recommends this approach when either parameter is near 1 or when the distribution is skewed, and our chosen parameterization of Beta(2.5, 5.0) is right-skewed, making the Gamma-ratio method effective for this case.

Explanation of Application to Selected Random Variables

We applied these theoretical algorithms to generate robust Gamma and Beta random variables in R-Studio. Our implementations utilized R's base generators (like rnorm()) because the acceptance-rejection method relies on drawing from simpler distributions to build complex ones.

To demonstrate functionality, we engineered a master routing function, `generate_gamma_complete(n, a)`. The function evaluates the shape parameter a and dynamically routes the execution to the correct sub-algorithm. If the user inputs $a = 0.5$, the code routes to Algorithm GS, applying the inverse-power transformations required for the singularity. If the user inputs $a = 2.0$, it routes to Algorithm GC,

generating baseline Cauchy proposals. If the user inputs $a = 100.0$, it uses Algorithm GO, leveraging the Gaussian squeeze bounds to maintain flat $O(1)$ execution time. Furthermore, we wrote a modular `generate_beta_from_gamma(n, a, b)` function that calls our Gamma suite twice and computes the resulting quotient.

To verify the functionality of our code, we generated $10,000$ samples across different parameters that correspond to each algorithm and a secondary Beta distribution ($a = 2.5, b = 5.0$). We plotted the histograms against the theoretical density curves. Then, we used each generated array in a Kolmogorov-Smirnov (K-S) statistical test. Comparing our empirical outputs against the theoretical cumulative distribution functions (`pgamma` and `pbeta`), all algorithmic modules yielded p-values well above the 0.05 significance threshold (meaning we fail to reject the null hypothesis). The statistical verification and visual alignment of the histograms stay consistent with the accuracy of the routing algorithm. All outputs pass as true Gamma and Beta variables.

Discussion of Implementation Challenges and Insights Gained

One of the first challenges we encountered was understanding why the threshold of $a = 2.53$ exists. At first glance it appears to be a tuning choice, but a careful reading of the paper revealed that it is the exact value where the cubic discriminant in Lemma 3 becomes positive. Below this threshold, the polynomial squeeze bounds used in Algorithm GO are not mathematically guaranteed to be valid lower bounds on the acceptance ratio $f(x)/g(x)$, meaning the algorithm could accept incorrect samples. This allowed us to understand that $a = 2.53$ is a mathematical boundary proven in Lemma 3. We gained insight on how limits in statistical software are dictated by rigid algebraic proofs, and implementing RNG algorithms without understanding those proofs makes it impossible to know whether the implementation is actually correct.

Another challenge came when we realized that only one of the introduced algorithms was not enough to generate a Gamma variable for all possible values of a. To bypass this, we had to figure out which algorithms to use and how to create the master function to combine them together. We found that Algorithm GS was used for values of a less than 1, Algorithm GC was useful for values of a greater than 1 and less than or equal to 2.53, and that Algorithm GO was useful for all values of a greater than 2.53. When creating our `generate_gamma_complete` master wrapper, we used if statements to verify that the correct algorithm was routed to and used based on the value of a. The insight that we gained from the challenge was an understanding of how foundational statistical libraries actually operate. We learned that software functions like R's `rgamma()` are not single equations but actually more sophisticated wrappers that take inputs and route them to specialized algorithms.

Appendix

Implementations of the master wrapper function as well as the three algorithm functions that the master uses.

```r
# Master Function: Dynamically routes shape parameter 'a' to the correct mathematical algorithm
generate_gamma_complete <- function(n, a) {
  if (a <= 0) stop("Shape parameter 'a' must be strictly positive (> 0)")

  if (a <= 1) {
    # Handles infinite asymptote at zero
    return(generate_gamma_GS(n, a))
  } else if (a <= 2.53278) {
    # Handles parameters where GO polynomial bounds are invalid
    return(generate_gamma_GC(n, a))
  } else {
    # Highly optimized Normal-Exponential split for large parameters
    return(generate_gamma_GO(n, a))
  }
}
```

```r
# Algorithm GS: Piecewise Envelope
generate_gamma_GS <- function(n, a) {
  samples <- numeric(n)

  # Step 1: Set b <- (e+a)/e
  b <- (exp(1) + a) / exp(1)

  for (i in 1:n) {
    accepted <- FALSE
    while (!accepted) {
      # Step 1: Generate u. Set p <- b * u
      u <- runif(1)
      p <- b * u

      # Step 1: If p >= 1 go to 3. (Meaning if p <= 1, proceed to Step 2)
      if (p <= 1) {

        # Step 2: (Case x <= 1). Set x <- p^(1/a). Generate u'.
        x <- p^(1 / a)
        u_prime <- runif(1)

        # Step 2: If u' > e^(-x) go back to 1. Otherwise deliver x.
        # (Evaluated on logarithmic scale to prevent floating-point precision loss)
        if (log(u_prime) <= -x) {
          samples[i] <- x
          accepted <- TRUE
        }
      } else {

        # Step 3: (Case x > 1). Set x <- -ln((b-p)/a). Generate u'.
        x <- -log((b - p) / a)
        u_prime <- runif(1)

        # Step 3: If u' > x^(a-1) go back to 1. Otherwise deliver x.
```

```r
        # (Evaluated on logarithmic scale to prevent floating-point precision loss)
        if (log(u_prime) <= (a - 1) * log(x)) {
          samples[i] <- x
          accepted <- TRUE
        }
      }
    }
  }
  return(samples)
}
```

```r
# Algorithm GC: Cauchy Envelope Method
generate_gamma_GC <- function(n, a) {
  samples <- numeric(n)

  # Step 1: Set b <- a-1, A <- 2a-1, and s <- sqrt(A)
  b <- a - 1
  A <- 2 * a - 1
  s <- sqrt(A)

  for(i in 1:n) {
    accepted <- FALSE
    while(!accepted) {
      # Step 2: Generate u. Set t <- s * tan(pi * (u - 0.5)) and x <- b + t.
      u <- runif(1)
      t <- s * tan(pi * (u - 0.5))
      x <- b + t

      # Step 3: If x < 0 go to 2.
      if(x >= 0) {

        # Step 4: Generate u'. If u' > exp(b * ln(x/b) - t + ln(1 + t^2/A)) go to 2.
        u2 <- runif(1)

        # Evaluated on a log scale to prevent exp() hardware overflow
        test_val <- b * log(x / b) - t + log(1 + (t^2 / A))

        if(log(u2) <= test_val) {
          # Step 4: Otherwise deliver x.
          samples[i] <- x
          accepted <- TRUE
        }
      }
    }
  }
  return(samples)
}
```

```r
# Algorithm GO: Modified Normal-Exponential Method with Squeeze Bounds
generate_gamma_GO <- function(n, a) {
  samples <- numeric(n)

  # Step 1: Set mu <- a-1, sigma <- sqrt(a + sqrt(8/3)*sqrt(a)), W <- sigma^2/mu,
  # d <- sqrt(6)*sigma, and b <- mu+d
```

```r
mu <- a - 1
sigma <- sqrt(a + sqrt(8/3) * sqrt(a))
W <- sigma^2 / mu
d <- sqrt(6) * sigma
b_bound <- mu + d
beta_const <- 0.009572265238289

for(i in 1:n) {
  accepted <- FALSE
  while(!accepted) {

    # Step 2: Generate u. If u <= 0.009572265238289 go to 8.
    u1 <- runif(1)

    if(u1 <= beta_const) {
      # TAIL REGION (Exponential Envelope)

      # Step 8: Take a sample s from the standard exponential distribution and set x <- b(1+s/d).
      s_exp <- rexp(1)
      x <- b_bound * (1 + s_exp / d)

      # Step 9: Generate u. If ln u > mu(2+ln(x/mu)-x/b) + 3.7203284924588 - b - ln(d/b) go to 2.
      u2 <- runif(1)

      # Exact logarithmic expansion applied dynamically to bypass textual rounding errors
      log_f <- mu * log(x / mu) - x + mu
      log_h <- mu * log(b_bound / mu) - (1 - mu / b_bound) * x
      log_r <- log(beta_const / (1 - beta_const)) + 0.5 * log(2 * pi * sigma^2) +
               mu * log(mu / b_bound) + log(d / b_bound) + d

      if(log(u2) <= log_f - (log_r + log_h)) {
        # Step 9: Otherwise deliver x.
        samples[i] <- x
        accepted <- TRUE
      }
    } else {
      # CORE REGION (Normal Envelope)

      # Step 3: Take a sample s from the standard normal distribution and set x <- mu + sigma*s.
      s_norm <- rnorm(1)
      x <- mu + sigma * s_norm

      # Step 3: If x < 0 or x > b go to 2.
      if(x >= 0 && x <= b_bound) {

        # Step 4: Generate u and set S <- s^2/2.
        u2 <- runif(1)
        S <- (s_norm^2) / 2

        # Lemma 3: Quick Acceptance Squeeze Bounds logic
        # Step 4: If s >= 0 go to 6.
        if (s_norm >= 0) {
```

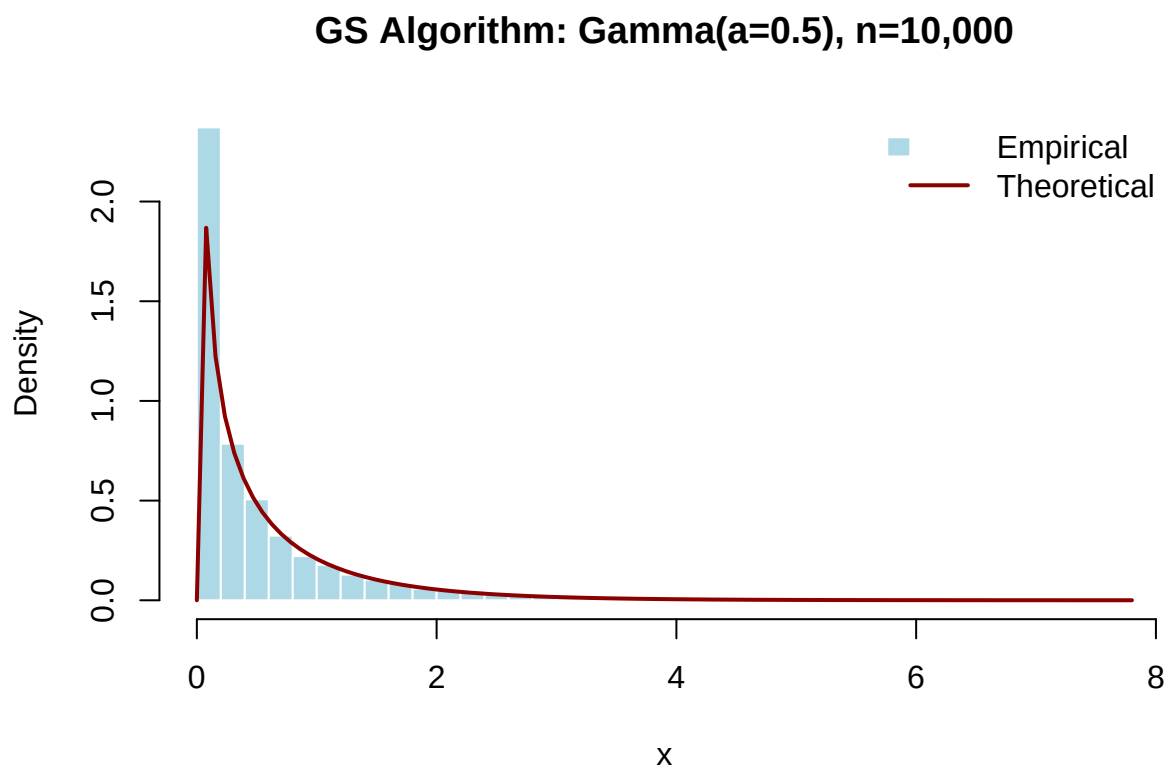
```r
            # Step 6: If u <= 1 - S(W-1) deliver x.
            L <- 1 - S * (W - 1)
          } else {

            # Step 5: If u <= 1 - S((1-2s/V)W-1) deliver x.
            # (Substituted with the explicit Lemma 3 polynomial from the text)
            L <- 1 - S * (W - 1) + (s_norm^3 / sqrt(a)) * W
          }

          if(u2 <= L) {
            samples[i] <- x
            accepted <- TRUE
          } else {
            # Step 7 (Full Evaluation): If ln u > mu(1+ln(x/mu)) - x + S go to 2, otherwise deliver x.
            if(log(u2) <= mu * (1 + log(x / mu)) - x + S) {
              samples[i] <- x
              accepted <- TRUE
            }
          }
        }
      }
    }
  }
  return(samples)
}
```

```r
# Beta Generation via Gamma Ratio (Ahrens & Dieter, 1974, Page 5)
generate_beta_from_gamma <- function(n, a, b) {

  # Step 1: Generate a sample x from a Gamma distribution gamma(a)
  x <- generate_gamma_complete(n, a)

  # Step 2: Generate a sample y from a Gamma distribution gamma(b), independent of x
  y <- generate_gamma_complete(n, b)

  # Step 3 & 4: Calculate the ratio x / (x + y). The resulting value follows a Beta(a,b) distribution
  beta_samples <- x / (x + y)

  return(beta_samples)
}
```

Empirical verifications of each algorithm. Here we will make use of plots and KS tests to make sure that our functions are correctly working to produce Gamma and Beta random variables.

```r
# Generate 10,000 samples for each segment
set.seed(42)
n_samples <- 10000

# Route through the master wrapper function
samples_GS <- generate_gamma_complete(n_samples, a = 0.5)
samples_GC <- generate_gamma_complete(n_samples, a = 2.0)
samples_GO <- generate_gamma_complete(n_samples, a = 10.0)

# Generate Beta samples using the Gamma Ratio method
samples_Beta <- generate_beta_from_gamma(n_samples, a = 2.5, b = 5.0)
```

```r
#Plots and KS tests for each algorithm:

#We make use of histograms of our functions as well as the curve of the theoretical function
#to make sure that our algorithms fit the shape of the random variable distributions.

# The K-S test evaluates if the empirical sample distribution significantly deviates
# from the theoretical CDF.
# A p-value > 0.05 indicates we fail to reject the null hypothesis.
# (Distributions match)

# Plot 1: Algorithm GS (a = 0.5)
hist(samples_GS, probability = TRUE, breaks = 50, col = "lightblue",
     main = "GS Algorithm: Gamma(a=0.5), n=10,000", xlab = "x", ylab = "Density", border = "white")
curve(dgamma(x, shape = 0.5, rate = 1), add = TRUE, col = "darkred", lwd = 2)
legend("topright", legend = c("Empirical", "Theoretical"),
       fill = c("lightblue", NA), border = c("white", NA),
       col = c(NA, "darkred"), lwd = c(NA, 2), bty = "n")
```

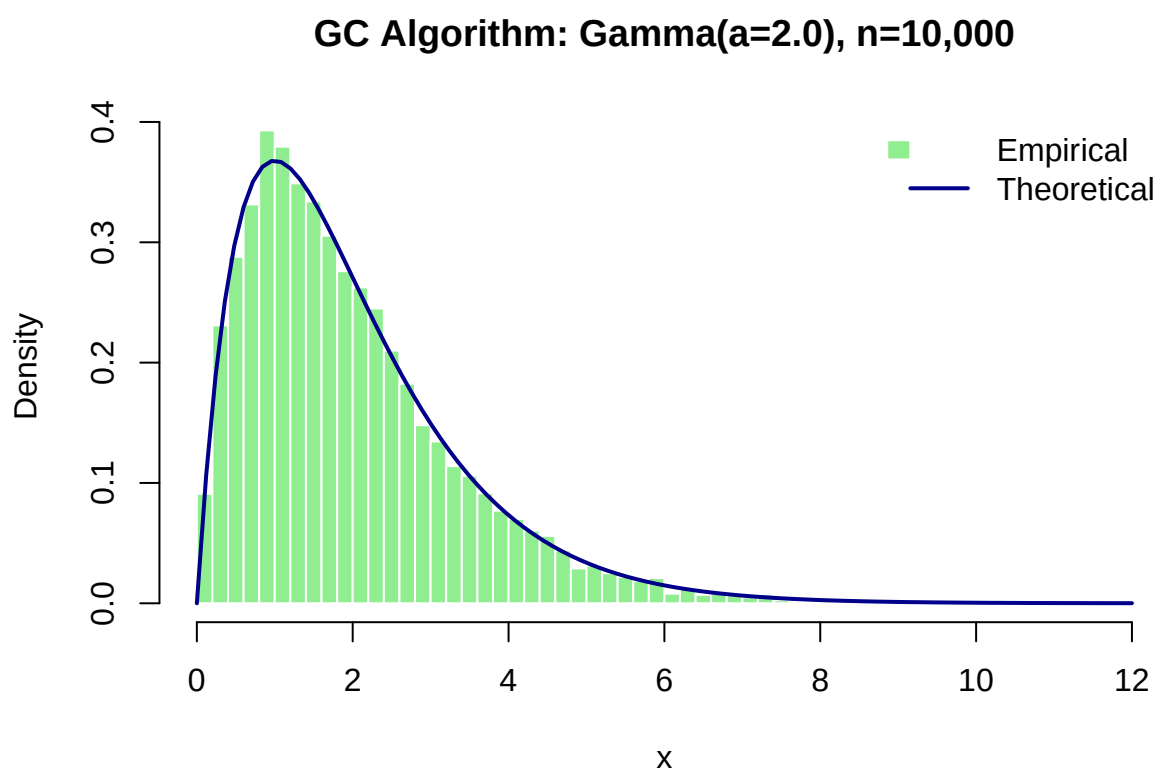



```r
# Test Algorithm GS (a = 0.5)
ks_GS <- ks.test(samples_GS, "pgamma", shape = 0.5, rate = 1)
cat("Algorithm GS (a=0.5) p-value:", ks_GS$p.value, "\n")
```

```
## Algorithm GS (a=0.5) p-value: 0.6150175
```

```r
# Plot 2: Algorithm GC (a = 2.0)
hist(samples_GC, probability = TRUE, breaks = 50, col = "lightgreen",
```

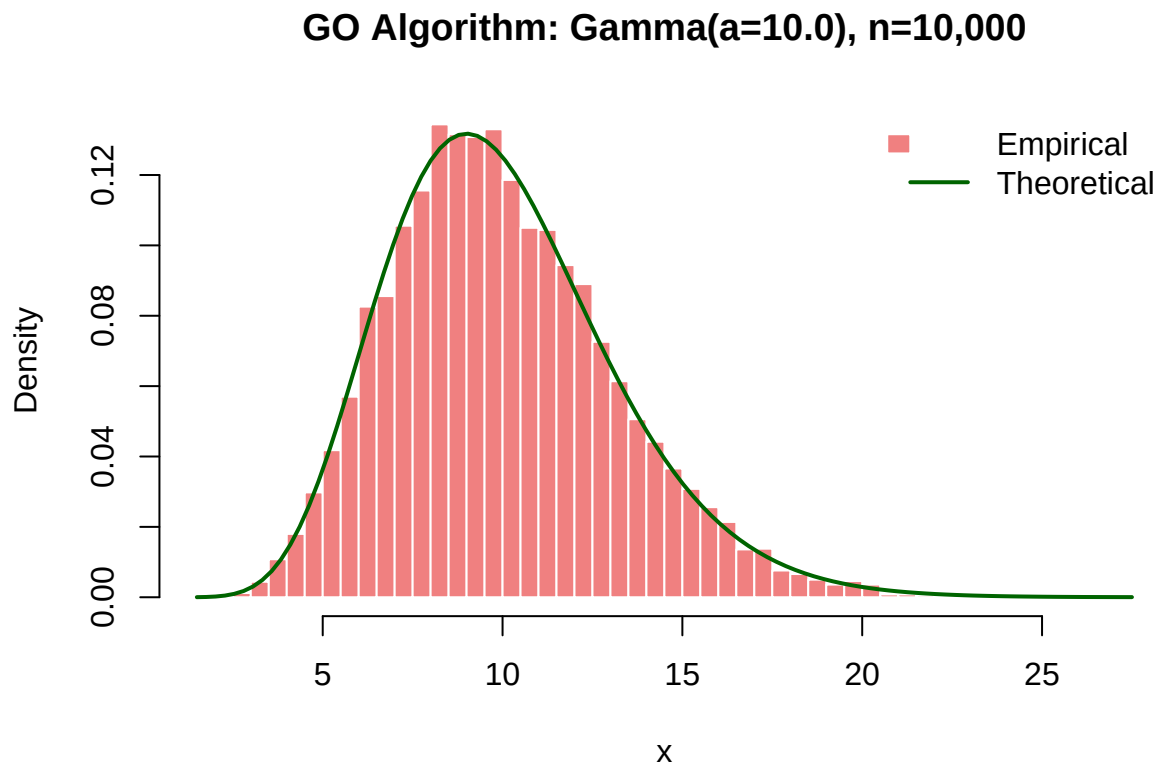
```r
     main = "GC Algorithm: Gamma(a=2.0), n=10,000", xlab = "x", ylab = "Density", border = "white")
curve(dgamma(x, shape = 2.0, rate = 1), add = TRUE, col = "darkblue", lwd = 2)
legend("topright", legend = c("Empirical", "Theoretical"),
       fill = c("lightgreen", NA), border = c("white", NA),
       col = c(NA, "darkblue"), lwd = c(NA, 2), bty = "n")
```




```r
# Test Algorithm GC (a = 2.0)
ks_GC <- ks.test(samples_GC, "pgamma", shape = 2.0, rate = 1)
cat("Algorithm GC (a=2.0) p-value:", ks_GC$p.value, "\n")
```

```
## Algorithm GC (a=2.0) p-value: 0.6292458
```

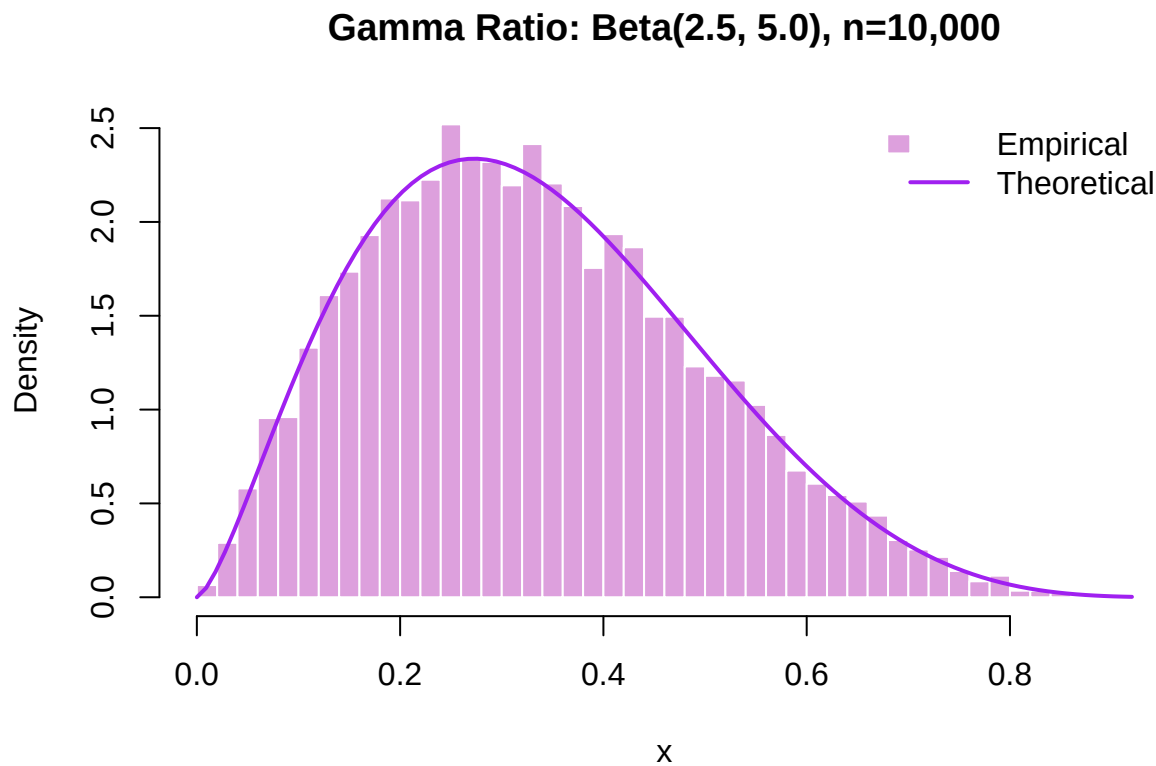
```r
# Plot 3: Algorithm GO (a = 10.0)
hist(samples_GO, probability = TRUE, breaks = 50, col = "lightcoral",
     main = "GO Algorithm: Gamma(a=10.0), n=10,000", xlab = "x", ylab = "Density", border = "white")
curve(dgamma(x, shape = 10.0, rate = 1), add = TRUE, col = "darkgreen", lwd = 2)
legend("topright", legend = c("Empirical", "Theoretical"),
       fill = c("lightcoral", NA), border = c("white", NA),
       col = c(NA, "darkgreen"), lwd = c(NA, 2), bty = "n")
```




```r
# Test Algorithm GO (a = 10.0)
ks_GO <- ks.test(samples_GO, "pgamma", shape = 10.0, rate = 1)
cat("Algorithm GO (a=10.0) p-value:", ks_GO$p.value, "\n")
```

```
## Algorithm GO (a=10.0) p-value: 0.5681361
```

```r
# Plot 4: Gamma-Ratio Beta Generator (a = 2.5, b = 5.0)
hist(samples_Beta, probability = TRUE, breaks = 50, col = "plum",
     main = "Gamma Ratio: Beta(2.5, 5.0), n=10,000", xlab = "x", ylab = "Density", border = "white")
curve(dbeta(x, shape1 = 2.5, shape2 = 5.0), add = TRUE, col = "purple", lwd = 2)
legend("topright", legend = c("Empirical", "Theoretical"),
       fill = c("plum", NA), border = c("white", NA),
       col = c(NA, "purple"), lwd = c(NA, 2), bty = "n")
```




```r
# Test Gamma-Ratio Beta Generator (a = 2.5, b = 5.0)
ks_Beta <- ks.test(samples_Beta, "pbeta", shape1 = 2.5, shape2 = 5.0)
cat("Gamma-Ratio Beta Generator (a=2.5, b=5.0) p-value:", ks_Beta$p.value, "\n")
```

```
## Gamma-Ratio Beta Generator (a=2.5, b=5.0) p-value: 0.7920064
```